

# Project Documentation: Predictive Maintenance & Anomaly Detection for IoT Fleets

## Table of Contents

1. Overview
  2. Business Value
  3. Architecture Diagram
  4. Technology Stack
  5. Project Structure
  6. Prerequisites
  7. Installation & Setup
  8. Running the Pipeline Step by Step
  9. Testing
    - Unit Tests
    - Integration Tests (Local)
  10. CI/CD Pipeline
  11. Troubleshooting
  12. Extending the Project
  13. References & Learning Resources
- 

## Overview

This project demonstrates an **end-to-end MLOps pipeline** for real-time anomaly detection in IoT sensor data. It simulates a fleet of vehicles sending telemetry (temperature, vibration, RPM) to Kafka, processes the stream with Apache Spark, stores raw data in HDFS and real-time anomalies in MongoDB, aggregates data nightly into PostgreSQL, trains an Isolation Forest model with MLflow, orchestrates retraining with Kubeflow, and serves predictions via a FastAPI endpoint.

The entire system is containerised with Docker Compose and designed for **Any** company specialising in AI, IoT, and smart city solutions.

---

## Business Value

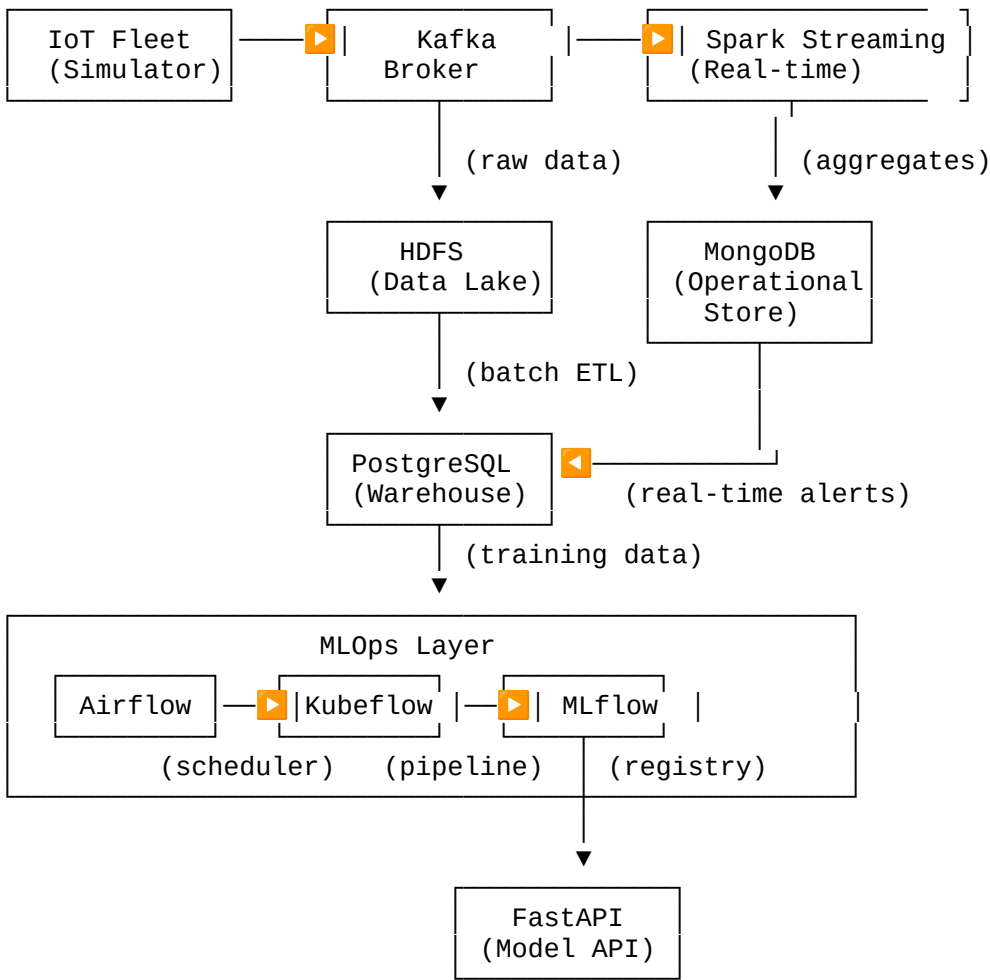
- **Real-time anomaly detection** – immediate alerts for critical failures (e.g., overheating, excessive vibration).

- **Scalable data ingestion** – Kafka and Spark handle millions of events per second.
- **Data lake + warehouse** – HDFS for long-term storage, PostgreSQL for business analytics.
- **MLOps governance** – MLflow tracks experiments, Kubeflow automates retraining.
- **Production-ready serving** – FastAPI/KServe provides low-latency predictions.
- **Full observability** – Prometheus + Grafana monitor pipeline health.

This architecture is directly applicable to Orion Valley’s smart city, logistics, and industrial IoT use cases.

## Architecture Diagram

text



Data flows from left to right; MLOps layer orchestrates model retraining on a schedule.

## Technology Stack

| Layer            | Technology               | Version        |
|------------------|--------------------------|----------------|
| Stream Ingestion | Apache Kafka + Zookeeper | 3.4.0 / latest |

| Layer                          | Technology                          | Version |
|--------------------------------|-------------------------------------|---------|
| Stream Processing              | Apache Spark (Structured Streaming) | 3.3.2   |
| Data Lake                      | Hadoop HDFS (Parquet)               | 3.2.1   |
| Operational DB                 | MongoDB                             | 6.0     |
| Warehouse                      | PostgreSQL                          | 14      |
| Workflow Orchestration         | Apache Airflow                      | 2.6.0   |
| ML Orchestration               | Kubeflow Pipelines                  | 1.7     |
| Experiment Tracking & Registry | MLflow                              | 2.3.2   |
| Model Serving                  | FastAPI                             | 0.95.0  |
| Monitoring                     | Prometheus + Grafana                | latest  |
| Containerisation               | Docker, Docker Compose              | 20.10+  |
| CI/CD                          | GitHub Actions                      | -       |
| Language                       | Python                              | 3.9     |

---

## Project Structure

text

```

orion-valley-predictive-maintenance/
├── .github/workflows/
│   └── ci.yml # CI pipeline (unit tests only)
├── data/
│   └── generator/
│       └── kafka_producer.py # Simulated IoT data source
├── processing/
│   ├── streaming/
│   │   └── streaming_anomaly_job.py # Spark streaming job
│   └── batch/
│       └── hdfs_to_postgres.py # Batch ETL (Airflow)
├── ml_pipeline/
│   ├── train_isolation_forest.py # Training with MLflow
│   └── kubeflow/
│       └── pipeline.py # Kubeflow pipeline definition
├── orchestration/
│   └── dags/
│       └── batch_etl_dag.py # Airflow DAG
├── serving/
│   └── api/
│       └── fastapi_app.py # Model serving API
├── monitoring/
│   └── prometheus.yml # Prometheus scrape config
├── tests/
│   ├── unit/ # Unit tests
│   └── integration/ # Integration tests (local only)
├── scripts/
│   └── setup_test_data.sh # Seed PostgreSQL
├── docker-compose.yml # All services
├── hadoop.env # HDFS config
├── requirements.txt
├── pytest.ini
├── Makefile
└── README.md

```

---

# Prerequisites

- **Docker** (20.10+) and **Docker Compose** (v2)
  - **Python 3.9+** with pip
  - **Java 11** (for Spark local mode)
  - **Git**
  - **8 GB RAM** recommended (for all containers)
- 

## Installation & Setup

### 1. Clone the repository

bash

```
git clone https://github.com/yourusername/orion-valley-predictive-maintenance.git
cd orion-valley-predictive-maintenance
```

### 2. Create a Python virtual environment

bash

```
python -m venv venv
source venv/bin/activate    # Linux/macOS
# or .\venv\Scripts\activate (Windows)
```

### 3. Install Python dependencies

bash

```
pip install -r requirements.txt
```

### 4. Start all services with Docker Compose

bash

```
docker-compose up -d
```

This starts:

- Zookeeper & Kafka
- Hadoop HDFS (Namenode + Datanode)
- Spark Master & Worker
- PostgreSQL
- MongoDB
- MLflow tracking server
- Airflow webserver & scheduler
- Prometheus

- Grafana

**Note:** Wait about 60 seconds for all services to fully initialise.

## 5. Verify services are running

```
bash
```

```
docker-compose ps
```

All services should show Up or running.

## 6. Seed test data (optional)

```
bash
```

```
bash scripts/setup_test_data.sh
```

---

# Running the Pipeline Step by Step

## Step 1: Generate simulated IoT data

```
bash
```

```
make produce
```

```
# or: python data/generator/kafka_producer.py
```

This script sends JSON messages to Kafka topic `iot-sensor-data` at 1 message per second (rotating among 100 vehicles).

## Step 2: Start the Spark streaming job

In a new terminal (with the virtual environment activated):

```
bash
```

```
make stream
```

```
# or: spark-submit --packages org.apache.spark:spark-sql-kafka-0-10_2.12:3.3.0 \
#       processing/streaming/streaming_anomaly_job.py
```

The streaming job:

- Reads from Kafka
- Computes 10-second sliding-window averages per vehicle
- Flags anomalies when average vibration > 3.0
- Writes raw data to HDFS (`hdfs://localhost:9000/user/raw_iot/`)
- Writes anomalies to MongoDB (`orion.anomalies` collection)

## Step 3: Train a model and log to MLflow

```
bash
```

```
make train
```

```
# or: python ml_pipeline/train_isolation_forest.py
```

- Loads historical data from PostgreSQL (table features)
- Trains an Isolation Forest model
- Logs parameters, metrics, and model to MLflow
- Registers the model as `OrionAnomalyDetector`

Visit <http://localhost:5000> to see the MLflow UI.

## Step 4: Serve the model via API

bash

```
make serve
# or: uvicorn serving.api.fastapi_app:app --reload --port 8000
```

Test the API:

bash

```
curl -X POST http://localhost:8000/predict \
  -H "Content-Type: application/json" \
  -d '{"temperature": 110, "vibration": 4.2, "rpm": 5800}'
```

Response: `{"anomaly": true, "score": -1}`

## Step 5: Trigger batch ETL (Airflow)

- Open Airflow UI at <http://localhost:8081> (user: `airflow`, password: `airflow`)
- Find DAG `orion_batch_etl_and_retraining`
- Trigger it manually (or wait for nightly 2 AM schedule)

This DAG:

1. Runs `hdfs_to_postgres.py` – aggregates daily data from HDFS to PostgreSQL
2. Triggers the KubeFlow pipeline (if deployed) to retrain the model

## Step 6: Monitor with Grafana

- Open Grafana at <http://localhost:3000> (user: `admin`, password: `admin`)
- Add Prometheus data source: `http://prometheus:9090`
- Import a dashboard (e.g., Kafka consumer lag, Spark metrics)

# Testing

## Unit Tests

Unit tests cover:

- Data generator logic
- Anomaly scoring function
- (No external dependencies – they run in CI)

Run locally:

```
bash
```

```
pytest tests/unit/ -v
```

## Integration Tests (Local)

Integration tests verify that Kafka, Spark, and MongoDB work together. They require all Docker services to be running.

```
bash
```

```
docker-compose up -d # ensure services are up
pytest tests/integration/ -v
```

**Note:** Integration tests are **not** executed in GitHub Actions due to Kafka startup reliability issues. They are fully functional locally.

---

## CI/CD Pipeline

The GitHub Actions workflow (`.github/workflows/ci.yml`) runs on every push and pull request:

- Python 3.9 setup
- Dependency installation
- Linting with `flake8`
- Unit tests with coverage reporting
- Coverage upload to Codecov (optional)

Integration tests are **disabled** in CI but documented for local execution.

---

## Troubleshooting

### Kafka doesn't start in Docker Compose

- Increase memory limit for Docker (minimum 4 GB).
- Use Bitnami images (already in `docker-compose.yml`).
- Wait longer – Kafka can take up to 60 seconds.

### Spark cannot connect to Kafka

- Ensure Kafka is up: `docker-compose ps`
- Check Kafka logs: `docker-compose logs kafka`
- Verify topic exists: `docker-compose exec kafka kafka-topics --list --bootstrap-server localhost:9092`

## MLflow fails to connect to PostgreSQL

- Ensure PostgreSQL is running: `docker-compose ps postgres`
- Check the connection string in `docker-compose.yml` for MLflow (uses `postgres` hostname, not `localhost`).

## HDFS commands fail

- Namenode may need extra time: `docker-compose logs hadoop-namenode`
- Use `hdfs dfs -ls /` inside the namenode container.

## FastAPI model not found

- Train the model first (`make train`)
  - Ensure the model file path in `fastapi_app.py` is correct (or load from MLflow registry).
- 

## Extending the Project

### Add a new data source (e.g., weather API)

- Create a new Kafka producer that pushes weather data
- Join it with vehicle data in Spark

### Replace Isolation Forest with a deep learning model (LSTM)

- Modify `train_isolation_forest.py` to use TensorFlow/PyTorch
- Log the Keras model with MLflow using `mlflow.tensorflow.log_model()`

### Deploy to Kubernetes (production)

- Use `kompose` to convert `docker-compose.yml` to K8s manifests
- Replace `localhost` references with Kubernetes service names
- Use KServe instead of FastAPI for serverless inference

### Add real-time dashboard (Streamlit)

- Read from MongoDB `orion.anomalies` every second
- Display live anomaly alerts and vehicle status

## License

MIT License – free to use, modify, and distribute for commercial and non-commercial purposes.